



NUI MAYNOOTH

Ollscoil na hÉireann Má Nuad

OLLSCOIL NA hÉIREANN MÁ NUAD
THE NATIONAL UNIVERSITY OF IRELAND
MAYNOOTH

SUMMER 2014 EXAMINATION

CS426

Computer Graphics

Dr. D. Charles, Dr. A. Winstanley, Dr. C. Markham

Time allowed: 2 hours

Answer ***three*** questions

All questions carry equal marks

[25 marks]

- 1 (a) Consider the following pov-ray project and answer the questions [9 marks] below. The picture shown is for a clock value of zero.

```

#include "colors.inc" } A
#include "math.inc"
#include "textures.inc" } B

background{ color Blue } } B

camera
{
    location <0, 1.5, -4.5> } C
    look_at <0, 0, 0>
}

light_source { <2, 4, -3> color White } } D

#declare lens=intersection
{
    sphere{<0.0, 0.0, 0.0>, 1.22}
    box{<-1.5,-1.5,1.1>,< 1.5,1.5,1.5>}
    pigment{ rgbf
    <1.0,1.0,1.0,0.95> }
    finish{phong 0.9 phong_size 10 reflection 0.1}
    interior{ior 1.5}
    translate<0,0,-1.1>
}
}

#declare source=union
{
    #if(clock<20)
    light_source{<0,0,0.8> color White} } F
    #end
    object{lens}
    cone
    {
        <0, 0, 1.2>,0
        <0, 0, 0>, 0.5
        open
        texture { SilverFinish }
    }
}
}

#declare torch=union
{
    object{source}

    cone
    {
        <0, 0, 1.3>, 0
        <0, 0,-0.1>, 0.6
        open
        pigment{ color Green }
    }

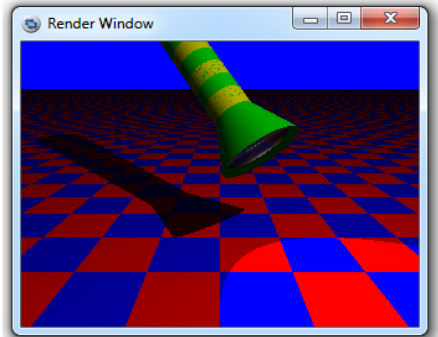
    #local tb=0;
    #while (tb<8)
    cylinder
    {
        <0,0,tb/4+0.4>,<0,0,tb/4+0.9>,0.3 open
        # if(odd(tb))pigment{ color Green }
        # else pigment {color Yellow} #end
    }
    #local tb=tb+1;
    #end
}
}

; POV-Ray animation of torch
Antialias=Off
Antialias_Threshold=0.1
Antialias_Depth=2
Input_File_Name="torch.pov"
Output_File_Name=c:\temp\
Initial_Frame=0
Final_Frame=30
Initial_Clock=0
Final_Clock=45
Cyclic_Animation=on
Pause_when_Done=off

```

Diagram illustrating the structure of the POV-Ray project file, with labels A through L grouping specific sections:

- A**: `#include "colors.inc"`
- B**: `#include "math.inc"` and `#include "textures.inc"`
- C**: Camera definition (`location` and `look_at`)
- D**: Light source definition (`light_source`)
- E**: Lens definition (`intersection` of sphere and box)
- F**: Conditional light source definition (`light_source` inside `union`)
- G**: Source definition (`union` of lens and cone)
- H**: Torch definition (`union` of cone and cylinder)
- I**: Cylinder definition (inside `while` loop)
- J**: Plane definition (`plane` with checkerboard texture)
- K**: Object definition (`object` with torch)
- L**: Animation settings (global options)



Continued on next page

- (a) Describe the overall function of each of the sections of code highlighted [A to L].

Clearly describe how sections [E],[G] and [H] generate a description of the main object in the scene.

Describe the animation produced when the (.INI) file is executed.

- (b) By default POV Ray uses a left-handed co-ordinate system. Explain this term and include a diagram with your answer. Make reference to the sign of the Z-axis moving into the screen and describe the rotation direction for increasing angle for each axis. [4 marks]

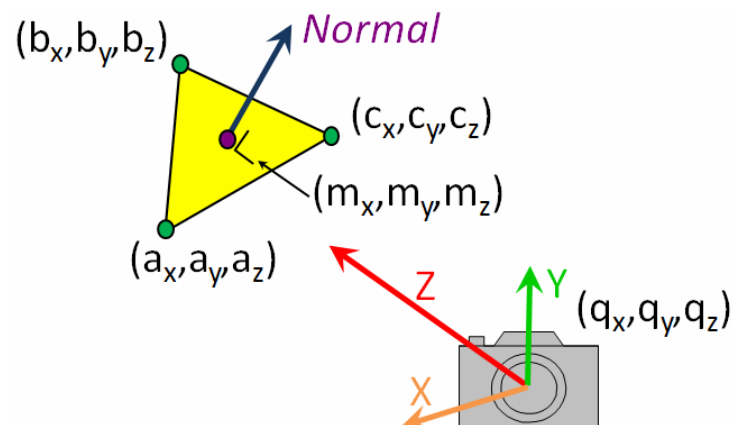
- (c) Clearly explain the term ray-tracing. In your answer distinguish between ray tracing and ray casting. Name and describe the different types of rays generated when ray tracing. Include a diagram with your answer. [6 marks]

- (d) Consider the following figure showing the geometry of a camera viewing a polygon (yellow triangle) and answer the following questions. [6 marks]

Explain how you would calculate the center of gravity, m , of the polygon.

Explain how you would calculate the surface normal to the polygon; you can assume a clockwise winding order.

Explain how you would determine if the camera is looking at the side of the polygon with the normal pointing towards it or away from it.



[25 marks]

- 2 (a) The rendering pipeline in a typical graphic system carries out the following transformations to display a group of 3D objects contained inside memory. Briefly describe each of the transforms in the following list. [5 marks]

- World transform
- View transform
- Projection transform
- Perspective divide
- Viewport transformation

- (b) Explain the term homogeneous co-ordinate. Write the following vector in homogeneous form; make reference to the scale factor, w , in your answer. Explain how the scale factor can be used to communicate position or direction information [7 marks]

$$\vec{v} = a \vec{i} + b \vec{j} + c \vec{k}$$

Describe the 4x4 matrices you would use to do each of the following.

- (i) translation of X,Y,Z by dx, dy, dz (ii) rotation θ about Z

Explain the term frame. Explain how the three vectors n, o and a are related in the matrix description of the frame shown below. Explain the information they carry. Include equations relating n, o and a . What information is carried by P_x, P_y and P_z ?

$$P = \begin{bmatrix} n_x & o_x & a_x & P_x \\ n_y & o_y & a_y & P_y \\ n_z & o_z & a_z & P_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- (c) Explain how to convert from RGB colour space to CMY. Use a calculation to identify the colour produced by printing cyan and magenta on the same region of white paper. Explain your reasoning. [4 marks]
- (d) Continued on next page ... [9 marks]

- 2 (d) Consider the following “cat and fish” game written for MonoGame/XNA and answer the following questions,
- (i) Describe the function of the highlighted sections of code [A to G].
 - (ii) Indicate the sequence of events that occurs when the program starts, executes and terminates.
 - (iii) Explain the game logic provided by the highlighted code [(i) to (iv)].
 - (iv) Describe how a player scores and how the game ends.

Continued on the next page....

2 (d) Continued ...

```
GraphicsDeviceManager graphics;
SpriteBatch spriteBatch;
```

} A (Explain function of each data structure)

```
Texture2D cat,fish;
SpriteFont font1;
int catpy=0, fishpx=350, fishpy=50,score=0,lives=3;
Random r=new Random();
```

} B

```
public Game1()
{
    graphics = new GraphicsDeviceManager(this);
    Content.RootDirectory = "Content";
    graphics.PreferredBackBufferHeight = 300;
    graphics.PreferredBackBufferWidth = 400;
}
```

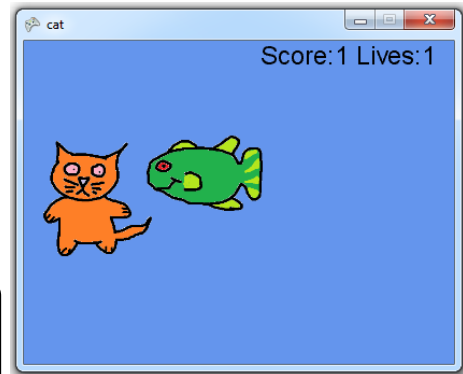
} C (Explain effect on run time display)

```
protected override void Initialize()
{
    base.Initialize();
}
```

} D

```
protected override void LoadContent()
{
    spriteBatch = new SpriteBatch(GraphicsDevice);
    cat = Content.Load<Texture2D>("cat");
    fish = Content.Load<Texture2D>("fish");
    font1 = Content.Load<SpriteFont>("Font1");
}
```

} E



```
protected override void Update(GameTime gameTime)
{
    KeyboardState keys = Keyboard.GetState();
    if (keys.IsKeyDown(Keys.Z)) catpy += 5;
    if (keys.IsKeyDown(Keys.W)) catpy -= 5;

    fishpx-=2;
    if (fishpx < -100)
    {
        fishpx = 450; fishpy = r.Next(250);
        lives--;
    }

    if((fishpx>-10)&&(fishpx<10)&&Math.Abs(fishpy-catpy)<20)
    {
        score++;
        fishpx = 450; fishpy = r.Next(250);
    }

    if (lives <= 0) this.Exit();

    base.Update(gameTime);
}
```

(i) (ii) (iii) (iv)

} F

```
protected override void Draw(GameTime gameTime)
{
    GraphicsDevice.Clear(Color.CornflowerBlue);
    spriteBatch.Begin();
    spriteBatch.Draw(cat, new Rectangle(0, catpy, 120, 120), Color.White);
    spriteBatch.Draw(fish,new Rectangle(fishpx, fishpy, 120, 120), Color.White);
    String text="Score:"+score.ToString() + " Lives:"+lives.ToString();
    spriteBatch.DrawString(font1,text, new Vector2(220, 0), Color.Black);
    spriteBatch.End();
    base.Draw(gameTime);
}
```

} G

[25 marks]

- 3 (a) Explain why the CIE introduced the concept of colour matching functions instead of using the spectral response of the R, G and B sensors of the human eye. Show how the colour matching functions relate to the Chromaticity Diagram and that colour is a two dimensional quantity. [6 marks]
- (b) Describe how a 3D model is described in a typical graphics system. In your answer make reference to the organization of the vertices and the use of (u,v) mapping. [4 marks]
- (c) A quaternion of the following form is regularly used in computer graphics problems. Explain this form of the quaternion; include a diagram to support your answer. [6 marks]

$$r = \left(\cos\left(\frac{\theta}{2}\right), \mathbf{u} \sin\left(\frac{\theta}{2}\right) \right)$$

Describe the quaternion operation that would be required to rotate a point $p=(0,p)$ about r . A full proof is not required.

- (d) The following code was created for MonoGame/XNA to explore methods for positioning and orientating and object. Consider the code and answer the following questions. [9 marks]
- (i) Explain how the highlighted lines of code [(i) to (iv)] setup and control the camera.
- (ii) Explain how the highlighted lines of code [A to G] control the position and orientation of the cube.
- (iii) Outline how you would modify the code so that the camera view can be controlled by the keyboard such that it rotates about the cube at a distance of five units about the Y-axis.

Continued on next page ...

3 (d) Continued

```

GraphicsDeviceManager graphics;
Model model;
Matrix[] boneTransforms;
public Matrix    view,projection; } (i)
public Vector3   cube_position;   } A
public Quaternion cube_orientation;

public Game1()
{
    graphics = new GraphicsDeviceManager(this);
    Content.RootDirectory = "Content";
    cube_position = new Vector3(0, 0, 0); } B
    cube_orientation = Quaternion.Identity;
}

protected override void LoadContent()
{
    model = Content.Load<Model>(@"cube");
    boneTransforms = new Matrix[model.Bones.Count]; } C
}

protected override void Update(GameTime gameTime)
{
    KeyboardState keys = Keyboard.GetState();
    if (keys.IsKeyDown(Keys.Left)) cube_position.X-=0.1f; } D
    if (keys.IsKeyDown(Keys.Right)) cube_position.X+=0.1f;
    // Y and Z ...

    if (keys.IsKeyDown(Keys.Q)) cube_orientation *= Quaternion.CreateFromYawPitchRoll(0.05f, 0, } E
    0);
    if (keys.IsKeyDown(Keys.W)) cube_orientation *= Quaternion.CreateFromYawPitchRoll(-0.05f, 0,
    0);
    // Pitch and Roll ...

    view = Matrix.CreateLookAt(new Vector3(0,0,5), new Vector3(0, 0, 0), Vector3.Up); } (ii)
    projection = Matrix.CreatePerspectiveFieldOfView(MathHelper.PiOver4,
        (float)graphics.GraphicsDevice.Viewport.Width } (iii)
        /(float)graphics.GraphicsDevice.Viewport.Height, 1.0f, 1000.0f);

    base.Update(gameTime);
}

protected override void Draw(GameTime gameTime)
{
    GraphicsDevice.Clear(Color.CornflowerBlue);
    model.CopyAbsoluteBoneTransformsTo(boneTransforms); } F
    foreach (ModelMesh mesh in model.Meshes)
    {
        foreach (BasicEffect effect in mesh.Effects)
        {
            effect.EnableDefaultLighting();
            effect.View = view; } (iv)
            effect.Projection = projection;

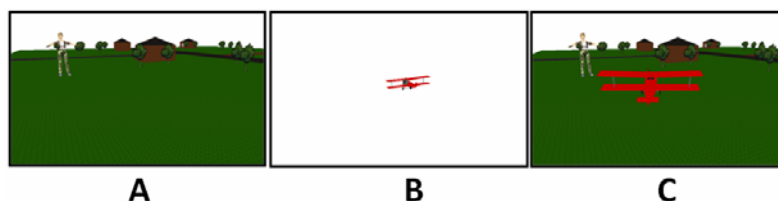
            effect.World = boneTransforms[mesh.ParentBone.Index]
                * Matrix.CreateFromQuaternion(cube_orientation) } G
                * Matrix.CreateTranslation(cube_position);
        }
        mesh.Draw();
    }
    base.Draw(gameTime);
}

```

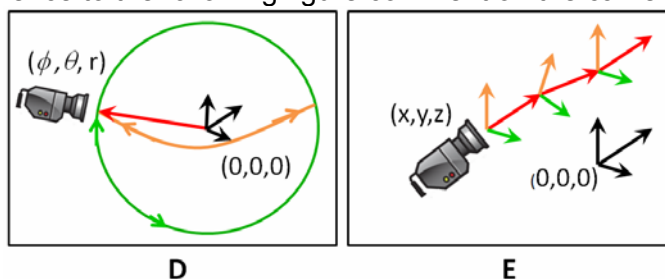


[25 marks]

- 4 (a) Describe the term fractal. Name three fractals that you have studied. [6 marks]
In the case of one of the fractals you mentioned outline the code (pseudo-code, C#, C or algorithm) that you would use to implement it. Comment on one application of fractal images in computer graphics.
- (b) Cameras can be described by their motion and their point of view. [4 marks]
With reference to the figure shown comment on the three points of view.



With reference to the following figure comment on the camera motion.



- (c) Describe the construction, the underlying physical principles and the operation of a single cell found in a typical LCD display. Comment on how a panel containing many pixels is achieved. Comment on how colour is achieved. Include a diagram with your answer. [6 marks]
- (d) With reference to the equations and code shown below answer the following questions. [9 marks]

$$m \frac{d^2 y}{dt^2} = mg - kv_y, v_y = \frac{dy}{dt}$$

$$m \frac{d^2 x}{dt^2} = -kv_x, v_x = \frac{dx}{dt}$$

What physical phenomenon is this program simulating? Describe the run time display.

Apply Euler's method to the equations above to generate the iterating equations that were used in the code. Note: they have been removed in the listing but a comment is in their place, $m=1$, $g=1$ and $k=0.025$.

With reference to the code shown below explain the terms double buffering and call back function. Refer to the relevant lines of code in the program.

Continued on next page ...

(d) Continued ...

```
#include "stdafx.h"
#include <freeglut.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>

float bally=20,ballvy=0;
float ballx=0, ballvx=12;
float dt=0.1;

static void display(void);
static void idle(void);

int _tmain(int argc, _TCHAR* argv[])
{
    glutInitWindowSize(512,512);
    glutInitWindowPosition(100,100);
    glutInit(&argc, (char **)argv );
    glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE | GLUT_DEPTH | GLUT_MULTISAMPLE);
    glutCreateWindow("Bounce OpenGL & FreeGlut");

    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0.0, 512.0, 512.0, 0.0);
    glClearColor(1,1,1,1);

    glutDisplayFunc(display);
    glutIdleFunc(idle);

    glutMainLoop();
    return 0;
}

static void idle(void)
{
    ballvy= // Asked in question
    bally= //      "

    ballvx= //      "
    ballx= //      "

    if (bally>500.0) ballvy=-ballvy;

    glutPostRedisplay();
}

static void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0,0.0,0.0);
    glRecti(ballx,(int)bally,ballx+10,(int)bally+10);
    glutSwapBuffers();
}
```

